

Testing de Software 2024
Trabajo Práctico Final

Github assignment: <https://classroom.github.com/a/tNRw7GgG>

Ejercicio 1: Se provee una implementación del juego threes (<http://threesjs.com/>). Específicamente se proveen clases que modelan (parcialmente) un tablero del juego.

- `ThreesBoard.java`: Modela el tablero del juego y es responsable de chequear que movimientos están habilitados en un determinado estado del tablero.
- `ThreesTile.java`: Modela una pieza del tablero.
- `ThreesController.java`: Esta clase es responsable de realizar los cambios correspondientes al tablero en cada movimiento.

a) Leer atentamente las reglas del juego y completar la implementación.

- Puede agregar los métodos que crea necesarios. Cada método agregado debe estar acompañado de un comentario JavaDoc.
- Si se encuentran errores en código ya provistos, estos deben ser claramente reportados y corregidos.
- Puede modificar el código fuente dado. Justifique en caso de introducir cambios.

b) La clase `ThreesBoard` debe estar acompañada de una *suite de test* construida utilizando alguno de estas 3 estructuras para modelar el software y algún criterio sobre ellas.

- Caracterización del dominio de inputs,
 - Grafos
 - Expresiones lógicas.
-
- Para el caso de Caracterización del Dominio de Inputs, piense características que involucren los métodos principales de la clase (pudiendo ignorar `toString()` y `getRandom()` por ejemplo).
 - Para el caso de grafos, tomar los métodos principales (pudiendo ignorar `toString()`, `getRandom()`, `setTile()`, `getTile()` y constructores) de la clase. Es posible elegir diferentes criterios en cada uno de ellos (aristas, pares de aristas, caminos principales). Puede utilizar la web para el cálculo de requisitos.

- Para el caso de Expresiones lógicas, tomar los métodos principales (pudiendo ignorar `toString()`, `getRandom()`, `setTile()`, `getTile()` y constructores), el criterio utilizado en estos casos deberá ser ACC en alguna de sus versiones.
- Se debe indicar el criterio y los requisitos de tests que deben ser cubiertos. Además, proveer la suite de test que cubren esos requisitos. La suite de test debe estar documentada, especificando el/los requisitos que cubre cada test.
- Especificar requisitos inviables (claramente).
- Utilizar JaCoCo para medir cobertura, no agregar test sino justificar el resultado obtenido.
- Utilizar cuando sea necesario *setUp*, *tearDown* y tests parametrizados.
- Utilizar nombres correctos para *suites de test* y *tests*.

Ejercicio 2: Provea una suite de tests de módulo/integración que logre alta cobertura de predicados de las acciones `moveUp()`, `moveDown()`, `moveLeft()` y `moveRight()`. Use JaCoCo para medir cobertura.

Nota: un test que efectúa dos movimientos diferentes en secuencia es un test de módulo.

- Utilizar cuando sea necesario `setUp`, `tearDown` y tests parametrizados.
- Utilizar nombres correctos para suites de test y tests.

Nota: Se provee un archivo `documentación.md` el cual deberá contener la documentación del trabajo práctico.

Sobre el Juego <http://threesjs.com>

El juego tiene lugar sobre un tablero de 4x4, alojando 16 piezas, también llamadas baldosas (tiles), que contienen números. El juego consiste en mover las piezas alrededor del tablero combinando piezas igualmente numeradas, y así crear nuevos números más grandes. El objetivo es obtener el mayor número posible combinando piezas. El juego finaliza cuando el tablero está completo, y no hay movimiento posible a realizar sobre el tablero.

Inicialización

Cada juego nuevo comienza con **9** piezas ubicadas aleatoriamente en el tablero. Cada pieza contiene un número entre: **1**, **2** y **3**.

Movimientos

Los movimientos posibles son hacia arriba, hacia abajo, a la izquierda y a la derecha. En cada movimiento, **todas las piezas** del tablero se mueven en la **misma dirección** a lo sumo un lugar.

Hay dos **excepciones** a esto:

1. las piezas linderas al borde del tablero, no podrán sobrepasarlo, por lo cual mantendrán su posición.
2. Si dos piezas se pueden combinar **y** se encuentran en una fila o columna linderas al borde del tablero, entonces las piezas serán sumadas. Este punto es fundamental para que las piezas puedan ser combinadas (sumadas), el cual constituye el objetivo principal del juego.

IMPORTANTE: En cada fila/columna sólo se realiza una combinación.

En cada movimiento, una nueva baldosa, numerada con 1, 2 o 3, ingresa a el tablero. La misma se inserta en alguna de las casillas opuestas a las filas/columnas modificadas en el último movimiento.

Reglas de combinación

Las piezas linderas numeradas con 3 o números más altos, sólo se pueden combinar si estas igualmente numeradas. Esto significa que 3 y 3 resultará en la pieza 6, pero 3 y 6 no pueden combinarse para obtener 9.

Las piezas numeradas con 1 y 2, se combinan para formar un 3. Si se tiene una pieza numerada con 2 es necesario una pieza numerada con 1, linderas a la anterior, para producir un 3.

Es posible hacer varias combinaciones en un solo movimiento, pero sólo para piezas que están en diferentes columnas o filas. Una fila con cuatro piezas numeradas con 3 no producirá una pieza numerada 12 en solo movimiento, sino que producirá un 6 y dos 3.

Invariante

Cada pieza que contenga un número **n** diferente a 1 y 2, puede descomponerse como el producto entre 3 y alguna potencia de 2, es decir, $\exists i \in \mathbb{N} : n = 3 * 2^i$